Loop Engineering

From Prompt Engineering to Autonomous Agent Loops

A practitioner's guide to designing self-running AI coding agents with Claude Code, Codex CLI, and the REAL/VEAL loop patterns for production multi-agent systems.

4 Levels v1.0 Last updated June 2026

loop-engineering agents claude-code codex autonomous REAL VEAL

On June 7, 2026, Peter Steinberger—creator of OpenClaw, the open-source AI agent that became the most-starred new repo in GitHub history—posted twelve words that broke the AI-coding internet:

> “You shouldn’t be prompting coding agents anymore. You should be designing loops that prompt your agents.”

The post hit 6.5 million views. Boris Cherny, creator of Claude Code at Anthropic, said the same thing: “I don’t prompt Claude anymore. I have loops that are running. They’re the ones that are prompting Claude and figuring out what to do. My job is to write loops.”

This is the shift from **prompt engineering** to **loop engineering**.

Prompt engineering optimizes a single interaction. Loop engineering designs systems that run agents in cycles—set a goal, take an action, observe the result, reflect, and repeat—until the goal is met or the system hands control back to a human.

This article covers the evolution from manual prompting to autonomous loops:

- **Built-in loops** in Claude Code (`/goal`, `/loop`) and Codex CLI (`/goal`)
- **The REAL pattern** for mission-driven, goal-oriented execution
- **The VEAL pattern** for validation-driven, state-convergent execution
- **Integration** between CLI tools and structured loop frameworks

Many articles discuss loops conceptually. This one shows how to actually build them.

The Problem with Prompting

Traditional AI coding workflows look like this:

```
Human types prompt
    ↓
AI generates response
    ↓
Human reviews output
    ↓
Human types next prompt
    ↓
Repeat
```

The human remains in the loop for every iteration. This is fine for exploration and learning, but it doesn't scale. If you're reviewing every AI response before the next step, you've replaced typing with reviewing—not fundamentally changed the workflow.

The insight from Steinberger, Cherny, and Anthropic's research team is that **the review step should be automated**. Instead of humans evaluating output, give the model a way to verify its own output. Close the loop.

Anthropic's research PM Theo describes this as the critical success factor:

> “Close the loop, give the model a way to verify its own output… The verification layer is equally important as generation capabilities—potentially more valuable for reliable deployment.”

Loop engineering shifts the human role from **operator** to **designer**. You design the loop once, then let it run.

Level 1: Built-in Loops in Claude Code and Codex CLI

Both Claude Code and Codex CLI now ship native loop primitives. These are the entry point to loop engineering.

Claude Code: `/goal`

The `/goal` command sets a completion condition and Claude keeps working toward it without you prompting each step.

```
/goal all tests in test/auth pass and the lint step is clean
```

How it works:

After each turn, a small fast model (Haiku by default) judges whether your condition is met
If not met, Claude starts another turn automatically
The goal clears when the condition is verified

Writing effective conditions:

The evaluator judges your condition against what Claude has surfaced in the conversation. It doesn’t run commands independently—it reads Claude’s output. So

write conditions as something Claude's own output can demonstrate:

- "All tests in `test/auth` pass" works because Claude runs the tests and the result lands in the transcript
- Include a stated check: "`npm test` exits 0" or "`git status` is clean"
- Add constraints: "no other test file is modified"
- Bound duration: "or stop after 20 turns"

Key insight: `/goal` uses a **separate evaluator model** to judge completion. The model that wrote the code doesn't grade its own homework.

Claude Code: `/loop`

The `/loop` command runs your prompt on a time interval.

```
/loop 5m
```

How it works:

- Claude executes the prompt
- Waits for the specified interval
- Runs again
- Continues until you press Esc or Claude decides the work is done

When to use each:

Command	Drives	Stops When	Best For
`/goal`	Progress toward finish line	Condition verified by evaluator	Completing discrete tasks
`/loop`	Time-based polling	Manual stop or self-determined	Monitoring external processes

As the Claude Code docs state: "/goal is for getting something done. /loop is for keeping an eye on something."

Codex CLI: `/goal`

OpenAI's Codex CLI shipped `/goal` in v0.128.0 (April 2026). Internally called the "Ralph Loop" after the iterative technique popularized by Geoffrey Huntley.

```
/goal implement user authentication with OAuth2 support
```

How it works:

The goal command enables Codex to enter an autonomous agentic loop:

Plan — Break the goal into executable sub-tasks
Act — Modify code, install dependencies, execute commands
Test — Run unit tests, linters, and builds
Review — Evaluate progress and identify blockers
Loop — Repeat until goal is met or blocked

Real-world example: Andrew Chen (a16z) left Codex running on a device driver project overnight. Fourteen hours later, it was still making progress with each iteration.

Cost consideration: Goal mode can dramatically increase token usage. Chen observed that `/goal` can "10,000x token use" compared to single prompts. Set budgets and monitor spend.

Level 2: The Limits of Built-in Loops

Built-in `/goal` and `/loop` commands are powerful, but they have constraints:

Single evaluator model. Both Claude Code and Codex use a single model or prompt to evaluate completion. For complex validation—build checks, multiple test suites, security scans, documentation—you need more structured evaluation.

No separation of concerns. The same agent writes code and judges its own work. As Addy Osmani (Google Cloud) notes: "The model that wrote the code is way too nice grading its own homework."

No escalation policy. When `/goal` gets stuck, it either keeps trying or stops. There's no structured way to escalate to a human, try a fallback agent, or abort with a useful error.

No multi-agent coordination. `/goal` runs a single agent. Production systems often need specialized agents—one to validate, one to fix, one to deploy.

No bounded execution. Without explicit limits, loops can burn tokens indefinitely. The `/goal` docs recommend including turn limits in conditions, but this is manual and error-prone.

These limitations point toward the next evolution: **structured loop patterns**.

Level 3: REAL Loops — Mission-Driven Execution

REAL (Read → Evaluate → Act → Loop) is a pattern for goal-oriented, mission-driven autonomous execution.

Unlike `/goal` which takes a simple condition, REAL loops operate on full mission specifications—roadmaps, PRDs, design documents—and work autonomously toward completion.

The REAL Pattern

```
Read
  ↓
Evaluate
  ↓
Act
  ↓
Loop
```

Read — The agent reads its mission specification and current environment:

- `ROADMAP.md`
- Product requirements
- Design specifications
- Repository source code
- Test results, lint reports, build logs
- Previous execution history

Evaluate — The agent assesses progress toward the goal:

- Which roadmap items remain?
- Which acceptance criteria are unmet?
- What is the highest-value next action?
- Has new work been discovered?

Act — The agent performs work:

- Writing or modifying code
- Running tests
- Updating documentation
- Creating pull requests
- Delegating to sub-agents
- Requesting human approval when necessary

Loop — The cycle repeats until:

- The mission is complete
- Success criteria are satisfied
- Human intervention is required
- Max attempts reached

REAL vs. REPL

The original **REPL (Read-Eval-Print Loop)** assumes a human is interacting:

REPL	REAL
Read user input	Read mission specification
Evaluate command	Evaluate progress toward goal
Print result	Act to change environment
Wait for next input	Loop until objective achieved

REPL is **input-driven**. REAL is **goal-driven**. The goal persists across iterations.

REAL Loop Definition

A REAL loop can be defined declaratively:

```yaml
name: feature-impl
type: REAL
description: Implement quarterly roadmap items
actor: developer
mission: |
  Implement all items in Q3 roadmap.
  Prioritize by business value.
  Create PRs for each feature.
max_attempts: 50
escalation: human
success_criteria: |
  All roadmap items completed or explicitly deferred.
  Each feature has passing tests and documentation.
```

Key fields:

- `actor` — The agent responsible for execution
- `mission` — The goal statement (what ROADMAP.md or `/goal` condition would contain)
- `max_attempts` — Bounded execution (typically 5-100 for REAL)
- `escalation` — What happens when max attempts reached: `human`, `abort`, `continue`, or `fallback`
- `success_criteria` — Clear definition of "done"

When to Use REAL

REAL loops are appropriate for:

- Feature implementation
- Codebase migrations
- Refactoring projects
- Roadmap execution
- Any open-ended work with a defined completion state

REAL loops **complete**. They terminate when the mission is achieved.

Level 4: VEAL Loops — Validation-Driven Convergence

VEAL (Validate → Evaluate → Act → Loop) is a pattern for state-driven, validation-focused execution with separated concerns.

The key insight: **separate the validator from the actor**. One agent checks state (read-only), another fixes issues (write access). This prevents the "grading your own homework" problem.

The VEAL Pattern

```
Validate
  ↓
Evaluate (GO / NO-GO)
  ↓
Act (if NO-GO)
  ↓
Loop
```

Validate — A read-only validator agent runs all checks:

- Build commands
- Test suites
- Linting
- Security scans
- Documentation completeness
- Pattern matching in files

Evaluate — Each check returns a status:

- `GO` — Check passed
- `NO-GO` — Check failed, needs fixing
- `WARN` — Passed with warnings

`SKIP` — Check skipped

Act — If any required check is NO-GO, the actor agent receives the findings and fixes the issues.

Loop — The validator re-runs checks. The cycle repeats until all checks pass or max attempts reached.

VEAL Loop Definition

```yaml
name: qa-fix
type: VEAL
description: QA validation and fix loop
validator: qa-agent
actor: code-fixer
max_attempts: 3
escalation: human
checks:
  - id: build
    type: command
    command: go build ./...
    required: true
  - id: tests
    type: command
    command: go test -v ./...
    required: true
  - id: lint
    type: command
    command: golangci-lint run
    required: true
  - id: security
    type: command
    command: gosec ./...
    required: false
```

Key fields:

`validator` — Read-only agent that runs checks
`actor` — Agent that fixes issues
`checks` — Array of validation checks with types:

`command` — Execute shell command, check exit code
`pattern` — Search for regex pattern in files
`file` — Check file existence/content
`manual` — Requires human judgment

`max_attempts` — Typically 3-5 for VEAL (if it can't fix in 3 tries, escalate)

VEAL vs. REAL

Aspect	REAL	VEAL
Category	Mission-driven	State-driven
Purpose	Build toward goal	Converge to valid state
Agent pattern	Single actor	Validator + Actor
Input	Mission specification	Validation checks
Typical attempts	5-100	3-5
Termination	Mission complete	All checks pass
Use cases	Features, migrations, roadmaps	QA fixes, lint fixes, docs updates

REAL creates. VEAL maintains.

REAL moves a system toward its desired state. VEAL keeps the system there.

When to Use VEAL

VEAL loops are appropriate for:

- QA validation and remediation
- Lint and format fixes
- Documentation completeness checks
- Security compliance validation
- Configuration validation
- Any bounded fix loop with idempotent checks

VEAL loops **converge**. They approach a valid state through iterative validation and correction.

Escalation Policies

Both REAL and VEAL loops need bounded execution. When max attempts are reached, four escalation policies control behavior:

Policy	Behavior
`human`	Stop and request human intervention
`abort`	Stop and fail the workflow
`continue`	Proceed despite unresolved issues
`fallback`	Invoke a fallback agent for resolution

Recommended max_attempts:

Loop Type	Range	Rationale
VEAL (QA fix)	3-5	If it can't pass QA in 3 attempts, there's likely a deeper issue
VEAL (docs fix)	3-5	Documentation updates should converge quickly
REAL (feature)	5-10	Features may need more iterations
REAL (roadmap)	50-100	Long-running work over many sessions

Integrating CLI Tools with REAL/VEAL

Can Claude Code's `/goal` and Codex's `/goal` be integrated with structured REAL/VEAL loops?

Yes. The integration happens at **three levels**:

1. Instructions-Based Integration

REAL and VEAL loop definitions can be compiled into agent instructions. When you generate agents for Claude Code, loop participation instructions are injected:

Validator agent receives:

- Loop participation section with role description
- Table of all validation checks
- Detailed check definitions
- Max attempts and escalation policy
- Instructions: "Run all checks, report GO/NO-GO status, do NOT modify files"

Actor agent receives:

Issues to address
Fix instructions
Max attempts and escalation policy
Instructions: “Fix identified issues, report actions taken”

Coordinator agent receives:

VEAL loop execution protocol (invoke validator → if NO-GO invoke actor → re-validate → loop)
REAL loop execution protocol (invoke actor with mission → check completion → loop)
Escalation handling instructions

The agents use Claude Code’s `/goal` internally, but operate within the structured VEAL/REAL protocol.

2. Hook-Based Validation

Claude Code supports hooks—shell commands that execute in response to events:

`before_file_write` — Validator can intercept writes (enforce read-only)
`after_command` — Capture validation results
`on_permission` — Control tool access per role
`on_subagent_stop` — Monitor agent lifecycle

Hooks enable **deterministic validation** alongside **LLM-based evaluation**. A VEAL loop’s command checks run via hooks, providing binary pass/fail signals.

3. MCP Server Integration

The Model Context Protocol (MCP) enables external tools to expose capabilities to Claude Code. A loop orchestration MCP server could:

Load REAL/VEAL loop definitions

Expose tools for running loop iterations

Track loop state across sessions

Handle escalation and fallback

Provide loop status to the coordinator agent

This enables **external orchestration** while Claude Code handles execution.

Practical Example: QA Fix Loop

Here's how a VEAL loop works in practice:

1. Define the loop:

```yaml
name: qa-fix
type: VEAL
validator: qa
actor: code-fixer
max_attempts: 3
escalation: human
checks:
  - id: build
    type: command
    command: npm run build
    required: true
  - id: test
    type: command
    command: npm run test
    required: true
  - id: lint
    type: command
    command: npm run lint
    required: true
```

2. Coordinator invokes validator:

```
Run all validation checks and report status.

Checks:
| ID    | Type    | Command          | Required |
|-------|---------|------------------|----------|
| build | command | npm run build    | Yes      |
| test  | command | npm run test     | Yes      |
| lint  | command | npm run lint     | Yes      |

Report GO or NO-GO for each check with findings.
```

3. Validator reports:

```
Validation Results:
- build: GO (exit 0)
- test: NO-GO (3 tests failing in auth.test.ts)
- lint: NO-GO (12 lint errors in src/components/)

Overall: NO-GO
```

4. Coordinator invokes actor:

```
Fix the following issues:

1. 3 tests failing in auth.test.ts
2. 12 lint errors in src/components/

Apply fixes systematically and report actions taken.
```

5. Actor fixes and reports:

```
Actions taken:
1. Fixed auth.test.ts: updated mock to match new API response shape
2. Fixed lint errors: added missing types, removed unused imports

Ready for re-validation.
```

6. Coordinator re-invokes validator:

```
Validation Results:
- build: GO
- test: GO
- lint: GO

Overall: GO
```

7. Loop completes successfully.

If still NO-GO after 3 attempts, escalate to human.

The Loop Engineering Mindset

Loop engineering requires a different way of thinking about AI coding:

From Prompts to Loops

Prompt Engineering	Loop Engineering
Optimize single interaction	Design repeating system
Human reviews each output	System verifies its own output
Manual iteration	Autonomous iteration
Human decides next step	Loop decides next step

Design Principles

1. Always include verification.

A loop with nothing to push back is the agent agreeing with itself on repeat. Include tests, type checks, linters, or other signals that can say "no."

As @mosyaseen noted in the viral thread: "Designing the loop is half of it. The other half is putting something in the loop that can say no."

2. Separate the validator from the actor.

The model that wrote the code is too nice grading its own homework. Use a separate evaluator—whether a different model, a deterministic check, or a specialized validation agent. See LLM-as-a-Judge for patterns on multi-judge aggregation and calibration.

3. Bound execution.

Without acceptance criteria, a budget cap, and a hard stop rule, a loop can burn tokens indefinitely while appearing productive. Set max_attempts. Include turn limits in conditions.

4. Define escalation.

What happens when the loop gets stuck? Human intervention, abort, continue anyway, or try a fallback agent. Make this explicit.

5. If you do something more than once, turn it into a loop.

Steinberger's recurring point: automate repetitive work. If you do something hard, turn it into a skill afterward so next time is free.

6. Don't over-engineer.

Building a loop for a task that runs once is waste, not automation. Loops have setup cost. Make sure the task is worth automating.

Cost Considerations

Loop engineering changes the economics of AI coding:

Token multiplication. Agent loops make many more model calls than single-shot prompts. A common rule of thumb is 10x to 100x more tokens.

Budget awareness. Set explicit token budgets. Include cost limits in loop definitions. Monitor spend per loop.

Time vs. tokens tradeoff. `/loop` with longer intervals (hourly, daily) reduces token use. `/goal` with tight completion conditions minimizes unnecessary iterations.

Evaluator efficiency. Claude Code uses Haiku (a small fast model) for `/goal` evaluation. This is much cheaper than using the main model for every judgment.

The Four Levels of Loop Engineering

To summarize the progression:

Level	Approach	Verification	Human Role
0	Manual prompting	Human reviews each output	Operator
1	Built-in `/goal`, `/loop`	Single evaluator model	Designer of conditions
2	Structured validation	Deterministic checks + LLM	Designer of check suites
3	REAL loops	Mission specifications, autonomous execution	Designer of missions
4	VEAL loops	Separated validator/actor, bounded convergence	Designer of validation systems

Most developers today operate at Level 0 or 1. The opportunity is in Levels 2-4, where the human designs systems rather than operating them.

Conclusion

The shift from prompt engineering to loop engineering represents a fundamental change in how we work with AI coding agents.

Prompt engineering optimizes individual interactions. **Loop engineering** designs autonomous systems.

The tools are converging on this pattern:

> Claude Code ships `/goal` and `/loop` as native primitives
> Codex CLI ships `/goal` with the Ralph Loop architecture
> Anthropic's internal teams run swarms of self-improving agents

The next evolution is **structured loop patterns**:

REAL for mission-driven, goal-oriented execution
VEAL for validation-driven, state-convergent execution
Integration between CLI tools and multi-agent frameworks

The engineers who thrive in this environment will be those who stop prompting and start designing loops.

As Boris Cherny said: “I feel that loops are the future. I strongly recommend them to anyone who hasn’t tried them yet.”

References

Steinberger, Peter. “You shouldn’t be prompting coding agents anymore.” X/Twitter. June 2026.
Cherny, Boris. “Coding Is Solved, and What Comes Next.” Square San Francisco. May 2026.
Anthropic. “Keep Claude working toward a goal.” Claude Code Documentation. 2026. code.claude.com/docs/en/goal
OpenAI. “Follow a goal.” Codex Use Cases. 2026. developers.openai.com/codex/use-cases/follow-goals
Willison, Simon. “Codex CLI 0.128.0 adds /goal.” April 2026. simonwillison.net/2026/Apr/30/codex-goals
Osmani, Addy. “Loop Engineering Best Practices.” Google Cloud. June 2026.
Huntley, Geoffrey. “The Ralph Wiggum Technique.” July 2025.
ProductBuildersHQ. “Software Delivery Autonomy Levels.” productbuildershq.com/frameworks/software-delivery-autonomy